DocsGenie — Complete Solution

> The full build, every file. Use it to check your work, not to skip the thinking. Read the story and the hints first.

The order below is the order to build in: the deterministic context and citation builders first (tested with no model), then the RAG layer, the grounded answer, and finally the pipeline that ties them together.

Setup files

`requirements.txt`: `langchain-openai`, `langchain-community`, `langchain-text-splitters`, `chromadb`, `openai`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` and `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

The sample docs (`docs/`)

A tiny product-documentation set to index. Save each as a real `.md` file.

```
# docs/installation.md
# Installing the SDK

Install the SDK from PyPI with pip:

    pip install acme-sdk

The SDK supports Python 3.10 and above. After installing, import it and
create a client with your API key to start making requests.
```

```
# docs/authentication.md
# Authentication

All API requests must be authenticated with an API key. Send the key in
the Authorization header as a Bearer token:

    Authorization: Bearer YOUR_API_KEY
```

```
Create and revoke keys in the dashboard under Settings > API Keys. Keep
your key secret; never commit it to source control.
```

```
# docs/billing.md
# Plans and Billing

The product has three plans: Free, Pro, and Enterprise. Webhooks and
priority support are included on the Pro and Enterprise plans. You can
change plan or update your payment method in the dashboard under Billing.
```

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    docs_dir: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
```

```
        docs_dir=os.getenv("DOCS_DIR", "docs"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The deterministic context and citation builders

Build these first. They are plain Python, so you can prove the grounding block and the citation list with hand-made passages — no model, no embeddings.

```
# src/context.py
# Turn retrieved passages into one model-ready block (each under a # Page:
# header) and the de-duplicated list of pages to cite. Plain Python, so
# it's fully testable with hand-made passages - no model or embeddings.
def build_context(passages):
    blocks = []
    for passage in passages:
        blocks.append(f"# Page: {passage['page']}\n{passage['text']}")
    return "\n\n".join(blocks)
```

```
def cited_pages(passages):
    pages = []
    for passage in passages:
        if passage["page"] not in pages:
            pages.append(passage["page"])
    return pages
```

The docs index and the retriever

```
# src/rag/docs_store.py
# Load the docs folder's .md pages, chunk them, keep each chunk's page
# name in its metadata (that name is the citation), embed, and store in
# Chroma.
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

import chromadb

COLLECTION_NAME = "documentation"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")

def build_docs_store(docs_dir):
    loader = DirectoryLoader(docs_dir, glob="**/*.md", loader_cls=TextLoader)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=60)
    chunks = splitter.split_documents(documents)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    texts = []
    metadatas = []
    for index in range(len(chunks)):
        ids.append(f"chunk-{index}")
        texts.append(chunks[index].page_content)
        path = chunks[index].metadata.get("source", "unknown")
        metadatas.append({"page": path.split("/")[-1]})
    collection.add(ids=ids, documents=texts, metadatas=metadatas,
```

```
                    embeddings=EMBEDDINGS.embed_documents(texts))
    return collection
```

```
# src/rag/retriever.py
# Return the doc passages closest to a question, each with its page name.
from src.rag.docs_store import EMBEDDINGS

def retrieve_passages(collection, question, k=4):
    if collection.count() == 0:
        return []
    results = collection.query(
        query_embeddings=[EMBEDDINGS.embed_query(question)], n_results=k,
    )
    documents = results["documents"][0]
    metadatas = results["metadatas"][0]
    passages = []
    for index in range(len(documents)):
        passages.append({
            "text": documents[index],
            "page": metadatas[index].get("page", "unknown"),
        })
    return passages
```

The grounded answer

```
# src/answer.py
# Answer the question using ONLY the retrieved documentation, so the
# answer is correct for THIS product - not the model's general knowledge.
from langchain_core.prompts import ChatPromptTemplate

ANSWER_PROMPT = (
    "You answer questions about a product using ONLY the documentation "
    "below. If the answer is not in it, say you don't know. Be specific "
    "and name the page(s) the answer comes from.\n\nDocumentation:\n"
    "{context}\n\nQuestion: {question}"
)

def answer_question(model, question, context):
    prompt = ChatPromptTemplate.from_template(ANSWER_PROMPT)
    chain = prompt | model
    reply = chain.invoke({"context": context, "question": question})
    return reply.content
```

Why grounding is the whole point:

- **"Using ONLY the documentation below"** keeps the answer tied to this product. Without it, the model falls back on how products are *usually* built — which may be nothing like this one.
- **"If it's not in it, say you don't know"** is the honesty rule: a question the docs don't cover should get "I don't know", not a plausible invention. Same discipline as every RAG answer (Doc08/10).
- The context handed in is `build_context(passages)` — one block with `# Page:` headers — so the model sees which passage came from which page.

The pipeline

```
# src/pipeline.py
# Index the docs, retrieve passages for the question, build the grounded
# context, answer, and return the answer plus the pages it came from.
from src.answer import answer_question
from src.context import build_context, cited_pages
from src.rag.docs_store import build_docs_store
from src.rag.retriever import retrieve_passages
from src.utils.logger import get_logger

def run_query(model, docs_dir, question):
    logger = get_logger(__name__)
    collection = build_docs_store(docs_dir)
    passages = retrieve_passages(collection, question)
    context = build_context(passages)
    answer = answer_question(model, question, context)
    pages = cited_pages(passages)
    logger.info(f"answered from {len(pages)} page(s)")
    return answer, pages
```

main.py

```
# main.py
import sys

from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_query

def main():
    if len(sys.argv) < 2:
```

```
        print('usage: python main.py "your question"')
        return
    question = sys.argv[1]

    config = load_config()
    model = get_model(config)
    answer, pages = run_query(model, config.docs_dir, question)

    print(f"\nAnswer:\n  {answer}")
    print("\nCited pages:")
    for page in pages:
        print(f"  - {page}")

if __name__ == "__main__":
    main()
```

The unit tests (no model, no embeddings)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "DOCSGENIE_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
```

```
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_context.py
# Run from the repo root: python -m tests.unit.test_context
from src.context import build_context, cited_pages

PASSAGES = [
    {"page": "authentication.md", "text": "Send an API key as a Bearer token."},
    {"page": "billing.md", "text": "Webhooks are on the Pro plan."},
    {"page": "authentication.md", "text": "Create keys in the dashboard."},
]

def test_every_passage_has_a_page_header():
    context = build_context(PASSAGES)
    result = "OK"
    if context.count("# Page:") != 3:
        result = "WRONG"
    if "# Page: authentication.md" not in context:
        result = "WRONG"
    print(f"case 1 (page headers in context) -> {result}")

def test_cited_pages_are_deduped():
    pages = cited_pages(PASSAGES)
    result = "OK"
    if pages != ["authentication.md", "billing.md"]:
        result = "WRONG"
    print(f"case 2 (cited pages deduped)     -> {result}")

if __name__ == "__main__":
    test_every_passage_has_a_page_header()
    test_cited_pages_are_deduped()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (page headers in context) -> OK
case 2 (cited pages deduped)     -> OK
```

The integration test (real model + RAG)

```
# tests/integration/test_live_query.py
# Real model + RAG. Run from the repo root:
#   python -m tests.integration.test_live_query
from src.config import load_config
from src.models.llm import get_model
from src.pipeline import run_query

def test_answers_from_the_docs():
    config = load_config()
    model = get_model(config)
    answer, pages = run_query(
        model, config.docs_dir, "How do I authenticate with the API?")
    ok = "key" in answer.lower() and "authentication.md" in pages
    status = "OK" if ok else "WRONG"
    print(f"case 1 (auth answered, page cited) -> {status}")

def test_offtopic_says_unknown():
    config = load_config()
    model = get_model(config)
    answer, pages = run_query(
        model, config.docs_dir, "How do I train a machine-learning model?")
    lowered = answer.lower()
    ok = "don't know" in lowered or "not" in lowered
    status = "OK" if ok else "WRONG"
    print(f"case 2 (off-topic -> unknown)      -> {status}")

if __name__ == "__main__":
    test_answers_from_the_docs()
    test_offtopic_says_unknown()
```

Expected run:

```
$ python main.py "How do I authenticate with the API?"

Answer:
  Authentication uses an API key sent in the Authorization header as a
  Bearer token. Create a key in the dashboard under Settings > API Keys.

Cited pages:
  - authentication.md
```

That's the whole project: the documentation indexed, the passages most relevant to the question retrieved, and the model answering only from them — with the pages it drew on cited, so the answer is both correct for this product and traceable back to the source.